

Multiobjective Simulated Annealing for Design of Combinational Logic Circuits*

Guoliang He

School of computer science
university of Wuhan
Wuhan, Hubei Province, China
guoliang.he@126.com

Yuanxiang Li

State Key Laboratory of Software Engineering
university of Wuhan
Wuhan, Hubei Province, China
yxli@whu.edu.cn

Xuan Wang, Wei Zhang and Zhifeng Dai

School of computer science
university of Wuhan
Wuhan, Hubei Province, China
wangxuan4376@163.com

Abstract - A multiobjective optimization technique was proposed for designing combinational logic circuits with 100% functionality and minimized number of gates. The main idea is to consider each output variant as an objective function, which is assigned to an individual. At first, it evolves each individual to satisfy the matches between the outputs produced by corresponding output variant of an encoded circuit and the values specified by the truth table. If an individual meets the above matches, it is further evolved to finish matches of the rest of the output variants. Once an individual is feasible, it is optimized in terms of the number of gates. Experiments are carried to assess the performance of multiobjective simulated annealing (MSA) against some intelligent algorithms and human designs, results illustrate MSA can design combinational logic circuits efficiently.

Index Terms - Combinational logic circuit, Evolvable hardware, Multiobjective optimization, Simulated annealing

I. INTRODUCTION

Evolvable Hardware (EHW) is a recently developed technology of circuit design which is built on the software-reconfigurable logic devices, such as PLD (Programmable Logic Device) and FPGA (Field Programmable Gate Array). These architectures represented by chromosome can be reconfigured mainly by using evolutionary computation. Nowadays, designing evolutionary circuits has become a research focus in this field and many methods were introduced[1-3].

Among them, design of combinational logic circuits is a branch. Kang Lishan proposed a new functional programmable device as EHW's structure[4]. Zhao Shuguang presented an intrinsic evolution technique in order to loosen some bottlenecks restricting the speed and scale of evolutionary design[5]. Kajitani proposed a variable length chromosome GA for design of EHW to improve the speed and scale of evolution of circuits[6]. Coello introduced some evolutionary algorithms to design combinational logic circuits, such as NGA (Genetic Algorithm with N-cardinality representation)[7] and MGA (Multiobjective Genetic Algorithm)[8]. NGA was a genetic algorithm with a matrix encoding scheme, and a n-cardinality alphabet. And they found this n-cardinality representation scheme more robust than the traditional binary representation used with GAs. MGA was population-based multiobjective optimization

technique such as VEGA (Vector Evaluated Genetic Algorithms)[9] to handle each of the outputs of a circuit as an objective. For example, a circuit with 4 inputs and 3 outputs would have $m=2^4 \times 3=48$ values to match. At each generation the population is split into $m+1$ subpopulations according to its corresponding output as its fitness (A subpopulation is added to consider the objective function).

In this paper, we use multiobjective simulated annealing (MSA) to design combinational logic circuits. Our proposal is to consider each output variant as an objective function, which is assigned to an individual. The number of individuals in MSA is the same as that of output variants in the true table. At first, this algorithm evolves each individual to satisfy the matches between the outputs produced by corresponding output variant of an encoded circuit and the values specified by the truth table. Then the individual is further evolved to finish matches of the rest of the output variants. Once the circuit is feasible, it is optimized in terms of the number of active gates. For the above example, this circuit would have $m=3$ individuals in the population and the main goal of each individual is to match its corresponding output variant with the truth table. At each generation, an individual is associated with each objective in terms of its fitness. In addition, an adaptive neighbourhood search method and a repair algorithm are also proposed to search a global solution with rapid convergence.

The remainder of the paper is organized as follows: section 2 elaborates on the circuit representation and a multiobjective simulated annealing algorithm to design combinational logic circuits. Section 3 gives some examples of its performance. Three methods including MSA, traditional simulated annealing and population-based simulated annealing are utilized to design combinational logic circuits in order to assess the advantage of MSA. And the results of MSA are compared against those produced by NGA, MGA and against human designs produced by KM (Karnaugh Maps)[10] and QM (Quine-McCluskey method)[11-12]. Last, there are conclusions and some possible paths of future research.

II. DESCRIPTION OF OUR APPROACH

A. Representation of Individual

We use a matrix to present combinational logic circuits[1], as shown in Figure 1. In order to represent combinational logic

* The research is supported by the National Natural Science Foundation of China under grant No. 60442001 and National High-Tech Research and Development Program of China (863 Program) No. 2002AA1Z1490.

circuits fully, the type of cells and the connectivity among cells are required to define for the matrix.

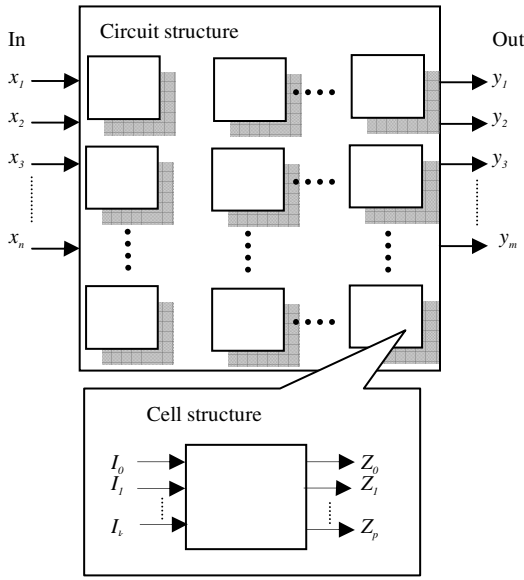


Fig.1 schematic of chromosome structure for combinational circuits

In the matrix, each cell is a gate with two inputs and one output. Here only five-type gates are used as follows: AND, NOT, OR, XOR and EQUATION, where EQUATION means the cell's input is directly connected to its output. For the connectivity, we define that only previous cells could connect to next cells to avoid the circle. And the degree of connectivity in the circuit called levels-back parameter determines how many columns to the left of current column can have their outputs connected to the inputs of the current cell. In this paper, the levels-back parameter defines one. So, each input of a logic gate may be connected to a primary input or the output of a previous logic gate in terms of the levels-back parameter. The outputs of the cells of the last column in the matrix are directly connected to the outputs of the encoded circuit respectively and the remaining cells of the last column are redundant.

B. Multiobjective Simulated Annealing Algorithm

The main idea behind our approach is to use a multiobjective optimization technique to handle each output variant of a circuit as an objective function, which is assigned to an individual. The number of the population is the same as that of the output variants. At each generation, according to the fitness of each individual for each output variant, an individual is assigned to each objective renewedly in terms of its fitness. At first, this algorithm evolves each individual to satisfy the matches between the outputs produced by corresponding output variant of an encoded circuit and the values specified by the truth table. If an individual meets the needs, it is further evolved to finish matches of the rest of the output variants. Once an individual is feasible, that is to say, the circuit satisfies the matches between all output variants and the truth table, the number of its gates is further minimized. Multiobjective simulated annealing is to design combinational

logic circuits with 100% functionality and minimized number of gates.

Here, a new adaptive mechanism for the neighbourhood search is proposed. On the one hand, if a combinational logic circuit is unfeasible, it may be impossible to evolve a valid circuit by changing too many gates. On the other hand, if the circuit is feasible, it is difficult to minimize the number of gates and remain its functionality by perturbing a gate (including its inputs and type). So, only one cell is perturbed before finding a valid individual, and more than one cell is changed in terms of the matrix size when the individual is feasible. In this way, it can search a global solution with rapid convergence.

In addition, a repair algorithm is implemented within the initialization and local search. In accordance with a gate (such as AND, OR and XOR) needing different inputs, the repair algorithm changes one of them to reduce the search space if the gate has the same inputs.

In SA, optimal choice of the parameters is very critical to the success of the SA method. The cooling schedule is the following form:

$T_{t+1} = \alpha T_t$, where α is called the cooling coefficient with $0.8 \leq \alpha < 1$.

C. Fitness Function

The fitness function is defined as the total number of matches by comparing outputs of an encoded circuit with values of the truth table (similar to [4]). In addition, another term is introduced considering the scale of circuit. The fitness function $F_f(x_i)$ evaluating an encoded circuit x_i is the following:

$F_f(x_i) = H(x_i) + w * V(x_i)$, where x_i is assigned to the objective of output variant I .

Here, $H(x_i)$ is defined by the degree of matching its corresponding output variant I with the truth table if the matches are not completed; otherwise it is defined by the degree of matching all output variants with the truth table. $V(x_i)$ is the total number of cells whose type are EQUATIONS in the circuit x_i and cells which are not active parts of this circuit despite their type being not EQUATIONS. w is weight coefficient, which changes in the different phases. If current circuit x_i is feasible, w is equal to 1; otherwise it's a small positive real number.

For example, an individual of a combinational logic circuit x_i with n inputs and m outputs, the method of computing $H(x_i)$ is as follow:

$$\text{If } H_f(x_i) < 2^n \quad H(x_i) = H_f(x_i) \quad \text{Else } H(x_i) = \sum_{j=1}^m H_j(x_i)$$

In this equation, $H_f(x_i)$ refers to the number of matches between the outputs produced by corresponding output variant I of an encoded circuit x_i and the values specified by the truth table. For a given combinational input value, if the match is successful, $H_f(x_i)$ is increased by one.

From fitness function, the first term can ensure that our algorithm explores along the direction of a valid circuit and the second term can be helpful to get the minimized circuit scale. The whole fitness function allows MSA to evolve a

combinational logic circuit with 100% functionality and minimized number of gates.

III. EXPERIMENTS

In order to assess the advantage of MSA, we use this algorithm to design three combinational logic circuits, which are enumerated as the true tables in Table 1. Symbol “In”

represents circuit inputs, and symbol “Out” represents outputs. On the one hand, three different simulated annealing algorithms (SAs) are compared to evaluate whether MSA is efficient or not. On the other hand, we compare MSA with other algorithms according to chromosome length and the number of gates.

Table 1 Truth tables for three designed circuits

Circuit no.1								Circuit no.2								Circuit no.3							
In				Out				In				Out				In				Out			
A ₁	A ₀	B ₁	B ₀	C ₁	C ₀	C ₁	C ₀	A	B	C	D	F ₁	F ₂	F ₃		A ₁	A ₀	B ₁	B ₀	C ₁	C ₀	C ₁	C ₀
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0		0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	1	0	1	0		0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	0	0	1	0	0	1	0		0	0	1	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	1	1	0	1	0		0	0	1	1	0	1	1	0
0	1	0	0	0	0	0	0	0	1	0	0	0	0	1		0	1	0	0	0	0	1	0
0	1	0	1	0	0	0	1	0	1	0	1	1	0	0		0	1	0	1	0	1	0	0
0	1	1	0	0	0	1	0	0	1	1	0	0	1	0		0	1	1	0	0	1	1	1
0	1	1	1	0	0	1	1	0	1	1	1	0	1	0		0	1	1	1	1	0	0	0
1	0	0	0	0	0	0	0	1	0	0	0	0	0	1		1	0	0	0	0	1	0	0
1	0	0	1	0	0	1	0	1	0	0	1	0	0	1		1	0	0	1	0	1	1	1
1	0	1	0	0	1	0	0	1	0	1	0	1	0	0		1	0	1	0	1	0	0	0
1	0	1	1	0	1	1	0	1	0	1	1	0	1	0		1	0	1	1	1	0	1	1
1	1	0	0	0	0	0	0	1	1	0	0	0	0	1		1	1	0	0	0	1	1	1
1	1	0	1	0	0	1	1	1	1	0	1	0	0	1		1	1	0	1	1	0	0	0
1	1	1	0	0	1	1	0	1	1	1	0	0	0	1		1	1	1	0	1	0	1	1
1	1	1	1	0	1	1	0	1	1	1	1	0	0	1		1	1	1	1	1	1	1	0

First, we use three SAs including MSA, traditional simulated annealing and population-based simulated annealing to design these logic circuits. Here traditional simulated annealing and population-based simulated annealing are the same except that the latter has more than one individual. In the neighbourhood search, these two algorithms use a static mechanism, which only perturbs one cell in the matrix through the process of evolution. During performed experiments, the parameters of SA are very critical to the success of getting an optimized solution. In order to compare MSA with other two SAs, we use the same parameters as follow: $T_0=0.08$, $\alpha=0.9$, the number of iteration $L_k=100$, maximum of number of

generations $T=50000$, number of SA runs $N=100$, $w=1$ or 0.01 . Each SA designs an identic circuit with the same matrix-size. The population size of MSA or population-based SA is the number of the circuit's output variants while that of traditional SA is one. By comparing percentage of 100% functional circuits, average number of generations required to obtain a feasible circuit, average number of generations required to obtain a optimized feasible circuit, maximum fitness of the optimized feasible circuit and percentage of maximum fitness, results obtained by MSA, traditional SA and population-based SA are summarized in Table 2. It can be noticed that MSA improves the performance of other two SAs.

Table 2 Results of designing three circuits with three simulated annealing algorithms

	Traditional SA			Population-based SA			Multiobjective SA		
	No.1	No.2	No.3	No.1	No.2	No.3	No.1	No.2	No.3
Percentage of 100% functional circuits	94	85	83	100	99	100	100	100	100
Average number of generations required to obtain a feasible circuit	2968	8741	12294	495	3628	5488	518	5790	3213
Average number of generations required to obtain a optimized feasible circuit	7136	17007	12294	1687	13297	5498	1996	16551	6032
Maximum fitness of the optimized feasible circuit	73	64	49	73	64	50	73	64	50
Percentage of maximum fitness	79	26	83	95	71	1	100	88	84

Then, we evaluate MSA in terms of chromosome length and the number of gates. If an algorithm can find same or better solution by either presenting an individual with shorter chromosome length or using the less number of gates in the design of EHW, we can say this algorithm is more adaptive for designing evolvable hardware. Here, we compare MSA with NGA, MGA, KM and QM from the point of above view.

The results obtained by NGA, MGA, KM, QM and MSA for circuits fulfilling the truth tables from Table 1 are presented in Table 3, 4 and 5. The obtained functions of MSA come from best fitness in the above experiment. Symbol

“Size” represents the size of the matrix structure, and symbol “N.G” represents the number of active gates in the circuit(sign “~” in function description represents negation). Results show MSA can design combinational logic circuits with shorter chromosome length, and is adaptive for design of evolvable hardware.

IV. CONCLUSION

In this paper we have presented a multiobjective simulated annealing algorithm to design combinational logic circuits at the gate level. The proposed approach is described and a few

examples of its use are presented. As compared with other two simulated annealing algorithms, it is easy to see that MSA can search a global solution with rapid convergence. At the same time, it also shows MSA is adaptive for designing evolvable hardware in terms of chromosome length. In future we want to explore the modification of the simulated annealing and the hybridization with other algorithms to design logic circuits.

Table 3 Comparison of the best solutions for Circuit no.1

Method	Obtained functions	Size	N.G
KM	$C_0 = A_0 \cdot B_0, C_1 = (A_0 \cdot B_1) \cdot (A_1 \cdot B_0)$ $C_2 = (A_1 \cdot B_1) \cdot (\sim(A_0 \cdot B_0)), C_3 = A_1 \cdot A_0 \cdot B_1 \cdot B_0$	—	8
QM	$C_0 = A_0 \cdot B_0$ $C_1 = (B_1 + B_0) \cdot (A_1 + A_0) \cdot ((A_1 \cdot A_0) \cdot (B_1 \cdot B_0))$ $C_2 = (A_1 \cdot B_1) \cdot (\sim(A_0 \cdot B_0)), C_3 = A_1 \cdot B_1 \cdot A_0 \cdot B_0$	—	12
NGA	$C_0 = A_0 \cdot B_0$ $C_1 = A_1 \cdot A_0 \cdot B_0 \cdot B_1 \cdot (A_0 \cdot B_1 + A_1 \cdot B_0)$ $C_2 = (A_0 \cdot B_0 + A_1 \cdot B_1) \cdot (A_0 \cdot B_0)$ $C_3 = A_1 \cdot B_1 \cdot A_0 \cdot B_0$	5×5	9
MGA	$C_0 = A_0 \cdot B_0, C_1 = (A_0 \cdot B_1) \cdot (A_1 \cdot B_0)$ $C_2 = (A_1 \cdot B_1) \cdot (A_0 \cdot B_0 \cdot A_1 \cdot B_1)$ $C_3 = A_0 \cdot B_1 \cdot A_1 \cdot B_1$	5×5	7
MSA	$C_0 = A_0 \cdot B_0, C_1 = (A_0 \cdot B_1) \cdot (A_1 \cdot B_0)$ $C_2 = A_1 \cdot (B_1 \cdot (A_1 \cdot B_0 \cdot A_0 \cdot B_1))$ $C_3 = A_1 \cdot B_0 \cdot A_0 \cdot B_1$	4×4	7

Table 4 Comparison of the best solutions for Circuit no.2

method	Obtained functions	Size	N.G
KM	$F_1 = (\sim(A \cdot C)) \cdot \sim(B \cdot D)$ $F_2 = \sim B \cdot D \cdot (\sim A + C) + (\sim A \cdot C)$ $F_3 = B \cdot \sim D \cdot (A + \sim C) + A \cdot \sim C$	—	19
QM	$F_1 = \sim(A \cdot C) \cdot \sim(B \cdot D)$ $F_2 = \sim A \cdot C + \sim(A \cdot C) \cdot (\sim B \cdot D)$ $F_3 = \sim(F_1 + F_2)$	—	13
NGA	$F_1 = \sim((B \cdot D) + (A \cdot C))$ $F_2 = ((B \cdot D) + (A \cdot C)) \cdot ((B \cdot D) + (A \cdot C)) \cdot \sim((D + (A \cdot C)) + \sim(\sim A + C))$ $F_3 = ((B \cdot D) + (A \cdot C)) \cdot (\sim(D + (A \cdot C)) + \sim(\sim A + C))$	6×7	12
MGA	$F_1 = \sim((B \cdot D) + (A \cdot C))$ $F_2 = F_1 \cdot ((B \cdot D) + (A \cdot C))$ $F_3 = ((B \cdot D) + (A \cdot C)) \cdot (A \cdot C) + (A \cdot B) \cdot (C)$	6×7	9
MSA	$F_1 = \sim((A \cdot C) + (B \cdot D))$ $F_2 = (A \cdot ((A \cdot C) + (A \cdot D))) \cdot ((A \cdot C) + (B \cdot D))$ $F_3 = ((B \cdot D) + (A \cdot C)) \cdot F_1$	5×5	9

Table 5 Comparison of the best solutions for Circuit no.3

method	Obtained functions	Size	N.G
KM	$C_0 = A_0 \cdot B_0$ $C_1 = (A_1 \cdot B_1) \cdot \sim B_0 + ((A_1 \cdot B_1) \cdot A_0) \cdot B_0$ $C_2 = A_1 \cdot B_1 + A_0 \cdot B_1 \cdot (A_1 + B_1)$	—	12
NGA	$C_0 = A_0 \cdot B_0, C_1 = A_0 \cdot B_0 \cdot (A_1 \cdot B_1)$ $C_2 = (A_0 \cdot B_0) \cdot (A_1 \cdot B_1) + (A_1 \cdot B_1)$	5×5	7
MGA	$C_0 = A_0 \cdot B_0, C_1 = A_0 \cdot B_0 \cdot (A_1 \cdot B_1)$ $C_2 = (A_1 \cdot B_1) + A_0 \cdot B_0 \cdot (A_1 \cdot B_1)$	5×5	7
MSA	$C_0 = A_0 \cdot B_0, C_1 = (A_1 \cdot B_1) \cdot (A_0 \cdot B_0)$ $C_2 = (A_1 \cdot B_1) \cdot (A_0 \cdot B_0) \cdot (A_1 \cdot B_1)$	3×3	7

REFERENCES

- [1] T. Kalganvoa and J. Miller, "Circuit layout evolution: an evolvable hardware approach," Colloquium on "Evolutionary hardware systems," IEE Colloquium Digest: London, UK, 1999.
- [2] Y.Y.H. Lam, "Simulated annealing based analog circuit synthesizer," Proceedings of IEEE TENCON '02, 2002, 659-662.
- [3] A.T. Soliman, H.M. Abbas, "Combinational circuit design using evolutionary algorithms," Electrical and Computer Engineering, IEEE CCECE 2003, 2003,1: 251-254.
- [4] Kang Li-Shan, He Wei, Chen Yu-Ping, "Evolvable hardware are realized with function type programmable device," Chinese Journal of Computers, 1999, 22(7):781-784.
- [5] Zhao Shu-Guang, Yang Wan-Hai, "Intrinsic hardware evolution based on prototype of function level FPGA," Chinese Journal of Computers, 2002,25(6):666-669.
- [6] Kajitani I, Hoshino T, Iwata M, Higuchi T, "Variable length chromosome GA for evolvable hardware," In: Proc the 1996 IEEE International Conference on Evolutionary Computation (ICEC'96), Piscatawa, NJ, USA, 1996:443- 447.
- [7] C. A. Coello, A. D. Christiansen, A. H. Aguirre, "Use of Evolutionary Techniques to Automate the Design of Combinational Circuits," International Journal of Smart Engineering System Design, 2000.
- [8] C. A. Coello Coello and A. H. Aguirre, "Design of combinational logic circuits through an evolutionary multiobjective optimization approach," Artificial Intelligence for Engineering, Design, Analysis and Manufacture, 2002, 16(1):39-53.
- [9] J.D. Schaffer, "Multiple Objective Optimization with Vector Evaluated Genetic Algorithms," Genetic Algorithms and their Applications: Proceedings of the First International Conference on Genetic Algorithms, Lawrence Erlbaum, 1985, 93-100.
- [10] M. Karnaugh, "A Map Method for Synthesis of Combinational Logic Circuits," Transaction of the AIEE, Communications and Electronic, 1953, 72(I):593-599.
- [11] W. V. Quine, "A Way to Simplify Truth Function," American Mathematical Monthly, 1955,62(9): 627-631.
- [12] E. J. McCluskey, "Minimization of Boolean Function," Bell Systems Technical Journal, 1956, 35(5): 1417-1444.